

实验：CoreMark 板端跑分

对应讲义 2.3 · 不重复 2.1 / 2.2 的 hello 与 Makefile 流程

实验目的

讲义 2.1–2.2 已完成交叉编译与工程构建；本实验在**同一工具链与 SSH 通道**上，把 CoreMark 跑到板端并解读结果。

- 练习：获取第三方 C 工程 → 交叉编译 → 部署 → 板端运行
- 理解：CoreMark 分数能说明什么、不能说明什么（见讲义 2.3）

前置：已完成 lecture.html 2.1、2.2；板端 SSH 可用；主机有 `riscv64-unknown-linux-gnu-gcc`。

操作步骤

1. 获取源码

从 EEMBC 官方或课程提供的镜像站下载 CoreMark，解压。阅读包内 `README`，确认使用 **Linux/POSIX 移植**（不是裸机 `baremetal` 移植）。

2. 交叉编译

在源码目录（具体目标名以该版本 Makefile 为准）：

```
$ export CC=riscv64-unknown-linux-gnu-gcc
$ make PORT_DIR=linux XCFLAGS="-O2" link
$ file coremark.exe
```

file 应显示 RISC-V ELF。若目标文件名不是 coremark.exe，以实际产物为准。

3. 部署到板端

```
$ scp coremark.exe user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/coremark.exe && ~/coremark.exe'
```

完整运行可能占用 CPU 数十秒至数分钟，属正常现象。

4. 记录输出

保存终端中的 Iterations、Total time、CoreMark 分数等关键行。

跑分期间勿拔电源。与他人共用板子时注意散热与占用时间。

结果记录与评述

记录表（自填）

项	你的结果
CoreMark 版本 / 源码来源	
编译命令摘要	

Iterations / Total time

CoreMark 分数

简答（各 2-4 句）

1. CoreMark 高分是否意味着 MQTT 或 DHT22 实验一定更流畅？为什么？
2. 列举两项影响你本次分数、但与「板子真实工作负载」无关的因素。
3. 与讲义 2.1 的 hello 相比，本实验在「工程流程」上多做了哪几步？（获取外部源码、移植选择等。）

完成标志

- 板端成功跑完并保存输出；
- 记录表与两道简答写完。